# Assignment Guide

10/09/2026 10:35:41
https://chatgpt.com/c/6ac865e8-eabc-83ee-a246-a86d08e2e3b7

Lab_4_VibeCoding_Student_handout.pdf my assignment link:...

Thinking ● Searched 10 websites ✓ Worked for 6s Okay, let's do this step by...

im done till opening the vs code and readme file

Thinking 🌐 Remembering ● Checked Lab 4 progress, project setup, GitHub fo...

Windows PowerShell Copyright (C) Microsoft Corporation. All rights reserved. P...

Player 2, move: V 1 0 Scores: P1=0 P2=1 | Turn: P1 ---. | X | ---. | . Player 1,...

Yes, Poojitha! We'll start recording before making any code changes. I'll tell yo...

i ran all these moves for the recording , before one Windows PowerShell...

Good, Poojitha! Your before recording shows the original game and the moves...

board.py game.py main.py README(20261009-041634).md requirements.txt...

Poojitha, I've received all your project files: board.py, game.py, main.py,...

Player 2, move: V 1 1 Player 2 completed 1 box(es) and plays again. Scores:...

Good, Poojitha! Your game is behaving correctly so far, and we've now...

Player 2, move: V 0 2 Player 2 completed 1 box(es) and plays again. Scores:...

Poojitha, we've finished testing most of the original gameplay. Your latest outp...

Player 1, move: H 2 1 Player 1 completed 1 box(es) and plays again. Scores:...

Thinking ● Inspecting the code I’m checking the board geometry, move...

ok gimme the moves to record and show the bug

Poojitha, I checked the moves you've already tried. They all produced correct...

Player 2, move: H 1 1 Scores: P1=0 P2=0 | Turn: P1 ---.--- | .--- | | . Player 1,...

Poojitha, don't worry! H 0 1 is being rejected because that horizontal edge is...

ok i want a bug

Okay, Poojitha. Let's find a real, reproducible bug in your original Dots and Box...

PS C:\Users\POOJITHA LAKSHMI\OneDrive\Desktop\23_dots_and_boxes> pytho...

Great, Poojitha! ✅ You reproduced a real bug in the original code. Your output...

yes i saved also stop calling my name everytime give the next step

Step 1 - Fix the invalid-coordinate bug Your before.mp4 is saved. Now we'll fix...

done i want to commit these changes and push

Step 1 - Check which files changed In your VS Code terminal, run: You should...

done pushing next what?

Thinking 🌐 Remembering ● Reviewed Lab 4 decisions, checked board.py,...

Its working

Great. Now commit and push the rematch feature as a separate commit for Ta...

done

Task 3 - Strengthen input validation We'll improve the existing input handling i...

DONE

Task 4 - Add automated tests Now we'll add automated tests for the game rule...

PS C:\Users\POOJITHA LAKSHMI\OneDrive\Desktop\23_dots_and_boxes> pytho...

All five automated tests passed, and your Task 4 commit has been pushed...

done

Lab_4_VibeCoding_Student_ha...

my assignment link:
[https://github.com/SETAPESU26/23_dots_and_boxes]
(https://github.com/SETAPESU26/23_dots_and_boxes)

1. fork
2. clone the forked repo
3. take a video
4. make changes and commit and push to ur repo
5. video after the changes

Deliverables :

on this repo :
[https://github.com/PoojithaBrpl/PES1UG24AM384_SE]
(https://github.com/PoojithaBrpl/PES1UG24AM384_SE)

submit 2 videos, this chat export, github repo link in LAB4 folder

guide me

*10/9/2026, 9:28:00 AM*

**Thinking**

- **Searched 10 websites**

✓ Worked for 6s

Okay, let's do this step by step. I'll guide you through the assignment from forking the repository to submitting everything in your `PES1UG24AM384_SE` repository.

I checked your **Lab 4: VibeCoding Student Handout**. There are a few important requirements beyond the five steps you listed:

- This is an **individual assignment** with a 60-minute duration.
- You must run the Python-based game before changing anything.

- Record a **10-second video before** making changes.
- Use a vibe-coding tool (such as ChatGPT) to fix broken code and implement the features listed in the assigned repository's README.
- Make **separate commits for each task**.
- Push the completed code to your fork, not to the original assignment repository.
- Submit the two videos, updated code, and exported chat history under the `Lab-4` folder in your lab repository.

## Our plan

**Step 1 - Fork the assignment repository**

```
  <text color="secondary" size="sm">Create your own copy on GitHub.</text>
</box>
<badge>First</badge>
```

**Step 2 - Clone and run the game**

```
  <text color="secondary" size="sm">Download your fork and verify the original
version works or identify its bugs.</text>
</box>
```

**Step 3 - Record the before video**

```
  <text color="secondary" size="sm">Capture 10 seconds before editing any code.
</text>
</box>
```

**Step 4 - Fix, implement, commit and push**

```
  <text color="secondary" size="sm">Use prompts to complete the README requirements,
with a separate commit per task.</text>
</box>
```

**Step 5 - Record the after video and submit**

```
  <text color="secondary" size="sm">Upload both videos, the chat export, and your
GitHub fork link to `Lab-4`.</text>
</box>
```

## Step 1: Fork the assignment repository

Do this first in your browser.

1. Open the original repository: 23_dots_and_boxes.
2. Make sure you're logged into your GitHub account.
3. Click **Fork** in the top-right corner.
4. On the next screen:
   - **Owner:** `PoojithaBrpl`
   - **Repository name:** `23_dots_and_boxes`
   - Leave the other settings at their defaults.
5. Click **Create fork**.

You should now have your own repository at:

`https://github.com/PoojithaBrpl/23_dots_and_boxes`

Make sure the page shows your username, `PoojithaBrpl`, and not `SETAPESU26`.

## Step 2: Clone your fork to your laptop

Once the fork is created:

1. Open your forked repository.
2. Click the green **Code** button.
3. Select **HTTPS** and copy the URL.
4. Open PowerShell and run:

If you copied a different URL, use that URL in the `git clone` command.

Now verify the files:

You should see your fork's URL for `origin`.

## Step 3: Read the README and run the original game

**Don't modify any code yet.** Your first video must show the original version before changes.

1. Open the project folder in VS Code:

   ```
   code .
   ```

2. Open `README.md` and check:

- How to install dependencies.
- How to run the game.
- Which bugs need fixing.
- Which features must be added.

3. Follow the README's instructions to run the game.
4. Record a **10-second video** showing the original game running. Save it somewhere easy to find, such as your Desktop, with the filename `before.mp4`.

If the game fails to run, don't silently fix it before recording. Capture the original state or error and share it with me so we can decide how best to demonstrate the before version.

## Step 4: Make changes using ChatGPT

Once we've checked the README, we'll handle the required fixes and features one by one.

For each task:

1. Identify the issue or required feature.
2. Use a clear prompt to implement it.
3. Run the game and test the change.
4. Save the changes in Git with a separate commit.
5. Push your work to your fork.

For example, after completing and testing an individual task:

Replace the commit message with a description of the actual task. If your repository uses a branch other than `main`, we'll adjust the command accordingly. Don't combine every task into one commit.

## Step 5: Record the after video

After all required fixes and features are implemented:

- Run the completed game.
- Demonstrate the new features and corrected behavior.
- Record another 10-second video and save it as `after.mp4`.
- Confirm your latest code has been pushed to GitHub.

## Step 6: Submit everything to your lab repository

Open your submission repository:

**PES1UG24AM384_SE**

Inside the `LAB4` folder specified in your assignment, upload these deliverables:

{@body const items = [{name:"before.mp4",desc:"10-second recording before changes"}, {name:"after.mp4",desc:"10-second recording after changes"}, {name:"Chat_History.pdf",desc:"Export of this ChatGPT conversation, including prompts and responses"},{name:"Updated project code",desc:"Completed code from your fork"}, {name:"GitHub repository link",desc:"Link to your forked 23_dots_and_boxes repository"}]} {@body const [checked,setChecked] = DIL.useState([])} **Submission checklist** {checked.length} of {items.length} {#each items as item} setChecked(old=>v? [...old,item.name]:old.filter(n=>n!==item.name))} lineThrough={false}> {item.name} {item.desc} {/each}

The handout explicitly lists the two videos, updated code, and exported chat history. Your GitHub link is an additional deliverable from the instructions you shared.

---

## Let's start now

**First, fork the repository.**

Open github.com/SETAPESU26/23_dots_and_boxes and click **Fork**.

When you've done that, tell me **"forked"**. I'll guide you through the next step without skipping anything.

*10/9/2026, 9:28:01 AM · gpt-6*

im done till opening the vs code and readme file

*10/9/2026, 9:32:58 AM*

**Thinking**

**Remembering**

**Checked Lab 4 progress, project setup, GitHub fork, and VS Code steps**

✓ Worked for 6s

Good! Now we'll continue from exactly where you are. Don't change any code yet — first, we need to understand how the game runs and record the original version. 😊

## Step 1: Check the README

In VS Code, read the `README.md` file and look for:

- Instructions to install dependencies.
- The command to run the game.
- Bugs that need fixing.
- Features you need to implement.

**Most importantly:** the README tells us what changes your assignment expects. We shouldn't guess which features to add.

## Step 2: Find the Python file

In the VS Code Explorer on the left:

1. Look for files ending in `.py`.
2. Also look for files such as `requirements.txt`.
3. Don't edit anything yet.

## Step 3: Send me what you see

Please send me a **screenshot of your README in VS Code**, showing the instructions and feature list.

I'll use that to tell you the exact commands to run in PowerShell, which Python file to execute, and how to record your 10-second **before video**.

We'll do this one step at a time, so you won't accidentally modify the original code before recording it.

*10/9/2026, 9:33:01 AM · gpt-6*

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\POOJITHA LAKSHMI\OneDrive\Desktop\23_dots_and_boxes> python main.py
Dots and Boxes
Enter moves as H row col or V row col.
Rows and columns start at 0.
Example: H 0 1

Scores: P1=0  P2=0 | Turn: P1
. 

. 

. 

Player 1, move:

*10/9/2026, 9:35:29 AM*

Player 2, move: V 1 0

Scores: P1=0  P2=1 | Turn: P1
---.
| X |
---.
|
   .

Player 1, move: H 2 0

Scores: P1=0  P2=1 | Turn: P2
---.
| X |
---.
|
---.

Player 2, move:

i still didnt start recording
now if i have to start , then tell me before u give any code

*10/9/2026, 9:39:14 AM*

Yes, Poojitha! We'll **start recording before making any code changes**. I'll tell you before every code change, as you requested. ❤️

One thing: the moves you've tried so far show correct box completion and scoring. We haven't identified the bug yet, so let's record the original game while we continue investigating.

## Step 1: Prepare the before video

Your game is still running in PowerShell, waiting at:

Don't close it.

### Record your screen on Windows

**1. Open Snipping Tool**

```
  Press the Windows key, type `Snipping Tool`, and open it.
</box>
```

**2. Select Record**

```
  Click the video camera/Record option, then click **New** and select the area
containing your PowerShell terminal.
</box>
```

**3. Start recording**

```
  Record approximately 10 seconds showing the original game and its current score and
board.
</box>
```

**4. Save the video**

```
  Save it on your Desktop as `before.mp4` (or keep the actual file extension Windows
provides).
</box>
```

## Step 2: Tell me when you're done

Once the video is saved, reply **"recorded"**.

Then we'll investigate the original bug, and I'll guide you through each required change, testing step, separate Git commit, and push. **I won't give you any code to modify until we've finished the before recording.**

*10/9/2026, 9:39:16 AM · gpt-6*

i ran all these moves for the recording , before one
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\POOJITHA LAKSHMI\OneDrive\Desktop\23_dots_and_boxes> python main.py
Dots and Boxes
Enter moves as H row col or V row col.
Rows and columns start at 0.
Example: H 0 1

Scores: P1=0  P2=0 | Turn: P1
  .

  .

  .

Player 1, move: H 0 0

Scores: P1=0  P2=0 | Turn: P2
---.

  .

  .

Player 2, move: V 0 0

Scores: P1=0  P2=0 | Turn: P1
---.
|
  .

  .

Player 1, move: H 1 0

Scores: P1=0  P2=0 | Turn: P2
---.
|
---.

```
  .

Player 2, move: V 0 1
Player 2 completed 1 box(es) and plays again.

Scores: P1=0  P2=1 | Turn: P2
---.
| X |
---.


  .

Player 2, move: V 1 0

Scores: P1=0  P2=1 | Turn: P1
---.
| X |
---.
|
  .

Player 1, move: H 2 0

Scores: P1=0  P2=1 | Turn: P2
---.
| X |
---.
|
---.

Player 2, move:
```

*10/9/2026, 9:45:41 AM*

Good, Poojitha! Your before recording shows the original game and the moves you've tested. ✅

But there's one important thing: **we haven't demonstrated the bug yet.** Your output shows that completing a box correctly increases Player 2's score and gives Player 2 another turn.

The README specifically asks for a before video showing the broken behaviour. We'll need to identify the bug before deciding whether your recording needs to be redone.

## What to do now

**First, save your current video** as `before.mp4`. Don't delete it.

Next, we need to inspect the original code to identify the actual bug rather than guessing.

In VS Code, look at the Explorer panel on the left and tell me which files you see in the project folder. In particular, check for files such as:

- `main.py`
- A file related to the board
- A file related to game logic or rules
- A `tests` folder

You can simply **send me a screenshot of the VS Code Explorer**, or paste the filenames here.

I'll guide you through investigating the bug next. We will not modify any code until we've identified the issue and confirmed what needs to be recorded.

*10/9/2026, 9:45:43 AM · gpt-6*

board.py

game.py

main.py

README(20261009-041634).md

requirements.txt

rules.py

*10/9/2026, 9:46:46 AM*

Poojitha, I've received all your project files: `board.py`, `game.py`, `main.py`, `rules.py`, `README.md`, and `requirements.txt`. I've inspected the code, and there's something important we need to address before changing anything.

## 1. What I found in your original code

Your project is divided into separate modules, as required by the assignment.

| File | Responsibility |
|---|---|
| `main.py` | Starts the game |
| `board.py` | Stores and displays the board and completed boxes |
| `game.py` | Manages turns, scores, and the game loop |
| `rules.py` | Validates moves and counts newly completed boxes |

In `game.py`, the scoring logic awards points for newly completed boxes and keeps the same player's turn when a box is completed. That matches the expected rules.

**I haven't yet established the exact intended bug from the source code alone.** Your recorded moves also showed correct scoring and turn behaviour for a completed box, so I don't want to invent a bug or give you a misleading demonstration.

## 2. What we should do next

Since you have already recorded the original gameplay, keep that video saved. Don't change any code yet.

Let's investigate the board-completion logic. In your VS Code terminal, the original game is still waiting for Player 2. You can finish the remaining moves to see whether the game correctly detects the end:

Type these commands one at a time:

Then:

Then:

These are intended to fill the remaining edges on the 2×2-dot board. If a command is rejected or the game ends earlier, stop and send me the output.

Once we see the result, we'll determine what actually needs fixing, choose a meaningful feature that satisfies the README, strengthen validation, and add automated tests.

**I will tell you explicitly before we make our first code change.**

*10/9/2026, 9:46:48 AM · gpt-6*

Player 2, move: V 1 1
Player 2 completed 1 box(es) and plays again.

Scores: P1=0  P2=2 | Turn: P2
---.
| X |
---.
| X |
---.

Player 2, move: H 0 1

Scores: P1=0  P2=2 | Turn: P1
---.---
| X |
---.
| X |
---.

Player 1, move: H 1 1

Scores: P1=0  P2=2 | Turn: P2
---.---
| X |
---.---
| X |
---.

Player 2, move:

*10/9/2026, 9:48:25 AM*

Good, Poojitha! Your game is behaving correctly so far, and we've now confirmed that Player 2 can complete both boxes and earn points.

Your latest output shows:

- Player 2 has scored 2 points.
- Player 1 has scored 0 points.
- The game is not finished yet because some edges remain undrawn.

## Step 1 - Finish testing the original game

At the current prompt, `Player 2, move:`, enter:

Then enter:

If either move is rejected, stop and send me the output. Otherwise, observe whether the game ends and announces the winner.

## Step 2 - What this tells us

We've tested box completion and scoring, but we still haven't demonstrated the specific bug described in the README. The source code alone hasn't established exactly which incorrect behaviour the assignment author intended.

**Don't change any code yet.** Once you finish these moves, I'll guide you through a targeted investigation of the original implementation before we start fixing it.

*10/9/2026, 9:48:28 AM · gpt-6*

Player 2, move: V 0 2
Player 2 completed 1 box(es) and plays again.

Scores: P1=0  P2=3 | Turn: P2
---.---
| X | X |
---.---
| X |
---.

Player 2, move: V 1 2

Scores: P1=0  P2=3 | Turn: P1
---.---
| X | X |
---.---
| X |   |
---.

Player 1, move:

*10/9/2026, 9:49:21 AM*

Poojitha, we've finished testing most of the original gameplay. Your latest output confirms that the scoring and extra-turn behaviour works for the moves we've tried.

But there's an important issue: **we haven't reproduced the bug yet**, and I don't want you to submit a video that claims to show a bug when it doesn't.

## Step 1 - Finish the current game

Your terminal is waiting for Player 1.

Enter:

Then enter the remaining valid moves until the board is full. If a move is rejected, stop and send me the output.

## Step 2 - Investigate the actual bug

From the files you uploaded, the likely area to investigate is the interaction between `board.py` and `game.py`:

- `board.py` tracks completed boxes and checks whether all lines have been drawn.
- `game.py` awards points and switches turns.

We need to test these behaviours systematically before deciding which code to change.

## Step 3 - Before we edit anything

I will explicitly tell you when we're about to make the first code change. Until then, don't replace any files or paste any new code.

Once we identify the bug, we'll complete the remaining assignment tasks: add a meaningful gameplay feature, improve input validation, write automated tests, document the changes, and create separate Git commits.

*10/9/2026, 9:49:24 AM · gpt-6*

Player 1, move: H 2 1
Player 1 completed 1 box(es) and plays again.

Scores: P1=1  P2=3 | Turn: P1
---.---
| X | X |
---.---
| X | X |
---.---

Game over!
Player 2 wins!
PS C:\Users\POOJITHA
LAKSHMI\OneDrive\Desktop\23_dots_and_boxes>

i want to find bug and show that in the before video

*10/9/2026, 9:50:48 AM*

**Thinking**

- **Inspecting the code**

  I'm checking the board geometry, move validation, box completion, scoring, and turn handling. The visible logic appears consistent so far, so I'm avoiding claiming a bug without a failing case.

- **Inspected the code**

- **Rechecking gameplay behavior**

  I'm testing edge cases such as completing multiple boxes, rejecting repeated lines, and ending the game. The supplied logic still appears correct, so I'm not finding a reproducible gameplay bug yet.

- **Searched 4 websites**

✓ Worked for 15s

Poojitha, now we've completed the entire original game, and I understand exactly what you want: **you need to reproduce the actual bug and capture it in your before video before fixing anything.**

Your final output shows:

- Player 1: 1 point
- Player 2: 3 points
- The game correctly detected completion and declared Player 2 the winner.

So the normal scoring, extra-turn, and end-of-game behaviour we tested all worked. We should not use that gameplay as proof of a bug.

The good news is that you uploaded the original Python files, so we can investigate the implementation directly instead of guessing.

## What I found after checking the original GitHub repository

I compared the code in your uploaded files with the public assignment repository. The scoring, box detection, and turn-switching logic appear to be implemented correctly in the current version. <Cite refs= {["turn754351view0","turn754351view1","turn754351view2"]}/>

There is, however, a **real validation weakness** in `board.py`: `Board.add_line()` assumes that the move has already been validated. If another part of the program calls it directly with an invalid orientation or out-of-range coordinates, it can modify the wrong structure or raise an exception. The normal gameplay path checks moves first, so this weakness does not explain the scoring behaviour you observed.

### What I recommend

Don't invent a scoring bug just to make the video. Your assignment asks you to investigate the broken code, so we should establish the actual failure first.

**Let's do one targeted check before editing anything.** Open `board.py` in VS Code and look at the `add_line()` method. You'll see that it writes directly into the board arrays without checking the coordinates itself.

We can use this finding to investigate the board-validation boundary, then implement a meaningful fix and test it. However, because this weakness isn't currently visible through ordinary gameplay, I can't honestly claim it is the specific bug your instructor intended.

Keep your existing `before.mp4` for now. We can record a more targeted demonstration once we establish a reproducible failure.

*10/9/2026, 9:50:52 AM · gpt-6*

ok
gimme the moves to record and show the bug

*10/9/2026, 9:53:05 AM*

Poojitha, I checked the moves you've already tried. They all produced correct scoring and turn behaviour, so I don't want to give you random moves and claim they'll show a bug.

Let's test a specific weakness in the original code instead. **This is a diagnostic test, not a confirmed reproduction of the instructor's intended bug.**

## Record a targeted test

Your game has finished, so you're back at the PowerShell prompt.

1. Open VS Code.
2. Open `board.py`.
3. Locate the `add_line()` method.
4. Don't edit anything yet.

The method writes directly to the board arrays without independently validating the move. Normal gameplay validates moves in `rules.py` first, so ordinary moves won't necessarily expose this weakness.

**There are no ordinary gameplay moves I can honestly promise will reproduce this particular issue.** We need to test the validation boundary directly rather than stage a false demonstration.

If your instructor specifically expects a gameplay bug in the original version, the next best step is to inspect the original repository's commit history or assignment notes to establish which failure was intentionally introduced. We can then reproduce that exact bug, record the 10-second video, and fix it.

*10/9/2026, 9:53:07 AM · gpt-6*

Player 2, move: H 1 1

Scores: P1=0  P2=0 | Turn: P1
```
---.---
   |
  .---
   |  |
  .
```

Player 1, move: H 0 1
Invalid or already-used move.

Scores: P1=0  P2=0 | Turn: P1
```
---.---
   |
  .---
   |  |
  .
```

Player 1, move:
H 0 1 IS NOT WORKING

*10/9/2026, 9:57:56 AM*

Poojitha, don't worry! `H 0 1` is being rejected because that horizontal edge is **already drawn** in your current game. The game is correctly reporting `Invalid or already-used move.`

Look at your board:

The top edge between the two middle dots is already present, so entering it again should be rejected.

### Let's reproduce a real bug

Your current output shows that repeated moves are rejected correctly during normal gameplay. We haven't yet found the intended bug, and I don't want you to record this as proof of a bug when the rejection is expected.

**Do this now:**

1. Stop the current game with `Ctrl + C`.

2. Don't edit any files.
3. Tell me whether you have saved your `before.mp4` recording.

We'll then investigate the original implementation systematically and identify a reproducible failure before recording the bug demonstration.

*10/9/2026, 9:57:58 AM · gpt-6*

ok
i want a bug

*10/9/2026, 9:58:40 AM*

Okay, Poojitha. Let's find a **real, reproducible bug** in your original Dots and Boxes project so your before video shows an actual failure.

From the code you uploaded, the ordinary gameplay tests haven't exposed a scoring bug. But I found a weakness worth investigating: `Board.add_line()` in `board.py` writes directly to the board without checking whether the coordinates are valid. The normal game validates moves beforehand in `rules.py`, so we need a targeted test to demonstrate the problem.

We won't edit the code yet. First, we'll reproduce the failure in the original version.

## Step 1 - Reproduce the bug

Open the VS Code terminal in your `23_dots_and_boxes` folder. You can leave the game stopped.

Start recording with Snipping Tool before running the test. Then copy and paste this **diagnostic command** into PowerShell:

```powershell
python -c "from board import Board; b=Board(); b.add_line('H', -1, 0); print('Top edge:', b.horizontal[0][0]); print('Bottom edge:', b.horizontal[-1][0])"
```

GenUI.copy("python -c \"from board import Board; b=Board(); b.add_line('H', -1, 0); print('Top edge:', b.horizontal[0][0]); print('Bottom edge:', b.horizontal[-1][0])\"")} variant="outline" color="secondary"> Copy command

### What this test demonstrates

The method receives row `-1`, which is an invalid coordinate, but Python interprets `-1` as the last row of a list.

The expected output is:

This demonstrates that the board can be corrupted when `add_line()` is called directly with an invalid coordinate. The regular gameplay path currently validates moves before calling this method, so this is a **module-level bug**, not a scoring bug we reproduced during normal gameplay.

## Step 2 - Save the recording

Stop recording after the output appears and save the video.

**Important:** This is useful evidence of a real weakness in the starter code, but it may not satisfy the instructor's requirement for a gameplay bug. We should keep that distinction clear.

Once you've run it, send me the output. Then we'll fix the validation issue properly, add the meaningful gameplay feature, write the required tests, and commit each task separately.

*10/9/2026, 9:58:47 AM · gpt-6*

PS C:\Users\POOJITHA LAKSHMI\OneDrive\Desktop\23_dots_and_boxes> python -c "from board import Board; b=Board(); b.add_line('H', -1, 0); print('Top edge:', b.horizontal[0][0]); print('Bottom edge:', b.horizontal[-1][0])"
Top edge: False
Bottom edge: True
PS C:\Users\POOJITHA LAKSHMI\OneDrive\Desktop\23_dots_and_boxes>

*10/9/2026, 10:02:09 AM*

Great, Poojitha! ✅ **You reproduced a real bug in the original code.**

Your output confirms it:

You passed row `-1`, which is an invalid coordinate. Instead of rejecting it, `Board.add_line()` modified the last row because Python allows negative list indices.

This is a genuine board-state validation bug. It is separate from the scoring behaviour we tested earlier.

## What to do now

**1. Save your before video** if you recorded the test. Keep it as `before.mp4`.

If you haven't recorded it yet, start Snipping Tool recording, run the same diagnostic command again, and capture the command and output. Aim for approximately 10 seconds.

**2. Don't change any code yet.**

Our next steps will be:

1. Fix the invalid-coordinate bug while preserving the modular structure.
2. Add a meaningful gameplay feature that interacts with the board and score system.
3. Strengthen input validation.
4. Create automated tests for the required game rules.
5. Document the changes and commit each task separately.
6. Record the after video and push the completed project.

**Tell me one thing: have you saved the before video now?** Once you confirm, I'll guide you through the first code change, explaining exactly which file to edit and what to replace.

*10/9/2026, 10:02:12 AM · gpt-6*

yes i saved
also stop calling my name everytime
give the next step

*10/9/2026, 10:03:00 AM*

## Step 1 - Fix the invalid-coordinate bug

Your `before.mp4` is saved. Now we'll fix the bug in `board.py` while preserving the project's modular structure.

The problem is that `add_line()` accepts a negative row index, allowing Python to modify the last row instead of rejecting the invalid coordinate.

### 1. Open `board.py` in VS Code

Find the existing `add_line()` method. Replace **only that method** with the following:

def add_line(self, orientation, row, col): if orientation not in {"H", "V"}: raise ValueError("Invalid orientation. Use H or V.")

```
    if orientation == "H":
        if not (0 <= row <= self.rows and 0 <= col < self.cols):
            raise ValueError("Invalid horizontal line coordinates.")
        self.horizontal[row][col] = True
    else:
        if not (0 <= row < self.rows and 0 <= col <= self.cols):
            raise ValueError("Invalid vertical line coordinates.")
        self.vertical[row][col] = True

    self._update_completed()
```

### 2. Save the file

Press **Ctrl + S**.

### 3. Test the fix

Run this command in the VS Code terminal:

This time, the program should report a `ValueError` for invalid coordinates instead of modifying the bottom row.

Now verify that a valid move still works:

Expected output:

If both tests behave as expected, tell me. Next we'll add the meaningful gameplay feature required by Task 2, then implement automated tests and separate commits for each task.

*10/9/2026, 10:03:05 AM · gpt-6*

done
i want to commit these changes and push

*10/9/2026, 10:05:29 AM*

### Step 1 - Check which files changed

In your VS Code terminal, run:

You should see `board.py` listed under modified files.

### Step 2 - Stage the change

### Step 3 - Commit the fix

This creates a separate commit for Task 1.

### Step 4 - Push to your GitHub fork

This assumes your current branch is `main`. If Git reports that the branch doesn't exist, run `git branch --show-current` and share the output.

### Step 5 - Verify the push

Open your fork: PoojithaBrpl/23_dots_and_boxes.

Refresh the page and check that `board.py` contains your change and that the latest commit message is **Fix invalid board coordinates**.

**Important:** This is only the first task's commit. We still need to implement the meaningful gameplay feature, strengthen input handling, add automated tests, and document the changes. Keep each task in its own commit as the assignment requires.

Run the commands above and tell me if any error appears.

*10/9/2026, 10:05:31 AM · gpt-6*

done pushing next what?

*10/9/2026, 10:06:45 AM*

**Thinking**

**Remembering**

- **Reviewed Lab 4 decisions, checked board.py, confirmed push, and listed README tasks**

✓ Worked for 11s

Next, we'll complete **Task 2: add a meaningful gameplay feature** from your assignment README.

We've already fixed the invalid-coordinate issue in `board.py` and you've pushed that change. We'll keep the project modular and make a separate commit for each task.

## Step 1 - Add a game replay/rematch feature

We'll add a **play-again option** after a game ends. This makes the game more complete because players can start another match without manually restarting the program.

Before editing, the plan is to:

- Keep the existing board, scoring, and turn logic.
- Add rematch handling in `game.py`.
- Reset the board and scores for a new match.
- Test that the second match starts cleanly.

## Step 2 - Edit `game.py`

Open `game.py` in VS Code. We are changing **only this file** for the rematch feature.

Replace the entire contents of `game.py` with this code:

from board import Board from rules import valid_move, completed_boxes

class DotsAndBoxes:
def **init**(self):

self.board = Board()
self.current = 0
self.scores = [0, 0]

```
def run(self):
    print("Dots and Boxes")
    print("Enter moves as H row col or V row col.")
    print("Rows and columns start at 0.")
    print("Example: H 0 1")

    while True:
        self._play_game()

        again = input("Play again? (Y/N): ").strip().upper()
        if again != "Y":
            print("Thanks for playing!")
            break

        # Reset the state for a fresh match.
        self.board = Board()
        self.current = 0
        self.scores = [0, 0]
        print("\nStarting a new game!")

def _play_game(self):
    while not self.board.is_complete():
        self.board.display(self.scores, self.current)
        raw = input(
            f"Player {self.current + 1}, move: "
        ).strip().upper()
        parts = raw.split()

        if len(parts) != 3:
            print("Invalid format. Use H row col or V row col.")
            continue

        orientation, row, col = parts

        if not row.isdigit() or not col.isdigit():
            print("Row and column must be non-negative numbers.")
            continue

        row, col = int(row), int(col)
```

```
        if not valid_move(self.board, orientation, row, col):
            print("Invalid or already-used move.")
            continue

        before = set(self.board.completed)
        self.board.add_line(orientation, row, col)
        newly_completed = completed_boxes(self.board, before)

        if newly_completed:
            self.scores[self.current] += newly_completed
            print(
                f"Player {self.current + 1} completed "
                f"{newly_completed} box(es) and plays again."
            )
        else:
            self.current = 1 - self.current

    self.board.display(self.scores, self.current)
    print("Game over!")

    if self.scores[0] == self.scores[1]:
        print("The game is a draw.")
    else:
        winner = 1 if self.scores[0] > self.scores[1] else 2
        print(f"Player {winner} wins!")
```

Save the file using **Ctrl + S**.

## Step 3 - Test the new feature

Run:

Play a complete game. When the game ends, you should see:

Type `Y` and press Enter.

The new game should start with:

- Scores reset to `P1=0, P2=0`
- Player 1 taking the first turn
- An empty board

Complete the second game and type `N` when prompted. The program should exit with `Thanks for playing!`.

**Don't commit this change yet.** First, test the rematch and tell me whether the board and scores reset correctly. Then we'll commit and push Task 2 separately.

*10/9/2026, 10:06:50 AM · gpt-6*

Its working

*10/9/2026, 10:11:39 AM*

Great. Now commit and push the rematch feature as a **separate commit for Task 2**.

## Step 1 - Check the changes

In your VS Code terminal, run:

You should see `game.py` listed as modified.

## Step 2 - Commit Task 2

Run these commands one at a time:

## Step 3 - Push to GitHub

Your second task is now committed separately from the coordinate-validation fix.

## Step 4 - Verify

Open your GitHub fork, refresh it, and check that the latest commit is `Add play again feature`.

## Next: Task 3 - Improve input validation

After pushing, we'll strengthen `game.py` so it safely handles malformed commands, invalid coordinates, repeated lines, and moves after the game is complete. Then we'll add the automated tests required by Task 4 and document the changes.

**First, finish the commit and push above.** Tell me when they're done.

*10/9/2026, 10:11:44 AM · gpt-6*

done

*10/9/2026, 10:12:29 AM*

## Task 3 - Strengthen input validation

We'll improve the existing input handling in `game.py` without changing the board display or scoring rules.

The assignment requires the game to handle:

- Malformed commands, such as `hello` or `H 0`.
- Invalid coordinates, including negative or out-of-range values.
- Repeated lines.
- Moves after the board is complete, without corrupting the board or score.

Your current game already handles several of these during normal gameplay. We'll make the validation more robust and then test each case.

### Step 1 - Edit `game.py`

Open `game.py` and find this section inside `_play_game()`:

Replace that section with:

if len(parts) != 3: print("Invalid format. Use H row col or V row col.") continue

```
        orientation, row_text, col_text = parts

        if orientation not in {"H", "V"}:
            print("Invalid orientation. Use H or V.")
            continue

        try:
            row = int(row_text)
            col = int(col_text)
        except ValueError:
            print("Row and column must be integers.")
            continue

        if row < 0 or col < 0:
            print("Row and column cannot be negative.")
```

```
            continue

        if not valid_move(self.board, orientation, row, col):
            print("Invalid coordinates or already-used move.")
            continue
```

Save using **Ctrl + S**.

## Step 2 - Test the validation

Run:

Try each input below at the move prompt.

| Input | Expected behaviour |
|---|---|
| `hello` | Invalid format |
| `H 0` | Invalid format |
| `X 0 0` | Invalid orientation |
| `H abc 0` | Integers error |
| `H -1 0` | Negative coordinates error |
| `H 99 99` | Invalid coordinates |
| `H 0 0` twice | Second move rejected |

For the repeated-move test, enter `H 0 0` once, then wait until the next player's prompt and enter it again. The second attempt should not change the board or score.

Also finish a game completely. Once the board is full, the game should display the result and ask whether to play again; it must not accept another move in the completed game.

## Step 3 - Commit and push Task 3

Once these tests pass, run:

This keeps Task 3 separate from your first two commits.

**After the push, tell me it's done.** We'll then create the automated tests required by Task 4, including horizontal moves, vertical moves, invalid/repeated moves, box completion, and the end-of-game condition.

*10/9/2026, 10:12:34 AM · gpt-6*

DONE

*10/9/2026, 10:14:11 AM*

## Task 4 - Add automated tests

Now we'll add automated tests for the game rules, as required by your README. We'll create a separate test file so the existing game code stays modular.

The tests will cover:

1. Valid horizontal move.
2. Valid vertical move.
3. Invalid and repeated moves.
4. Completing a box and awarding a point.
5. Detecting when the board is full.

We'll use Python's built-in `unittest` module, so you don't need to install any extra packages.

## Step 1 - Create the test file

In VS Code Explorer:

1. Right-click your project folder, `23_dots_and_boxes`.
2. Select **New Folder** and name it `tests`.
3. Right-click `tests` → **New File**.
4. Name it `test_game.py`.

Your project structure should look like this:

## Step 2 - Paste the test code

Open `tests/test_game.py` and paste this entire code:

import unittest

from board import Board
from rules import valid_move, completed_boxes

class TestDotsAndBoxes(unittest.TestCase):

```
def setUp(self):
    self.board = Board()

def test_valid_horizontal_move(self):
    self.assertTrue(valid_move(self.board, "H", 0, 0))
    self.board.add_line("H", 0, 0)
    self.assertTrue(self.board.horizontal[0][0])

def test_valid_vertical_move(self):
    self.assertTrue(valid_move(self.board, "V", 0, 0))
    self.board.add_line("V", 0, 0)
    self.assertTrue(self.board.vertical[0][0])

def test_invalid_and_repeated_moves(self):
    self.assertFalse(valid_move(self.board, "X", 0, 0))
    self.assertFalse(valid_move(self.board, "H", -1, 0))
    self.assertFalse(valid_move(self.board, "H", 99, 0))

    self.board.add_line("H", 0, 0)
    self.assertFalse(valid_move(self.board, "H", 0, 0))

    with self.assertRaises(ValueError):
        self.board.add_line("H", -1, 0)

def test_box_completion(self):
    self.board.add_line("H", 0, 0)
    self.board.add_line("H", 1, 0)
    self.board.add_line("V", 0, 0)

    before = set(self.board.completed)
    self.board.add_line("V", 0, 1)

    newly_completed = completed_boxes(self.board, before)

    self.assertEqual(newly_completed, 1)
    self.assertIn((0, 0), self.board.completed)

def test_end_of_game_condition(self):
    self.assertFalse(self.board.is_complete())

    for row in range(self.board.rows + 1):
        for col in range(self.board.cols):
            self.board.add_line("H", row, col)
```

```
    for row in range(self.board.rows):
        for col in range(self.board.cols + 1):
            self.board.add_line("V", row, col)

    self.assertTrue(self.board.is_complete())
```

if **name** == "**main**":
unittest.main()

Save the file with **Ctrl + S**.

## Step 3 - Run the tests

In your VS Code terminal, make sure you're in the main project folder, then run:

You should see five tests, all marked `ok`, followed by:

The exact runtime may differ. If any test fails, send me the complete terminal output before changing anything else.

## Step 4 - Commit and push Task 4

If all five tests pass, run:

That creates a separate commit for Task 4.

## Step 5 - Document your changes

After the tests pass, we'll update `README.md` to document:

- The invalid-coordinate fix.
- The play-again feature.
- The improved input validation.
- How to run the automated tests.

Then we'll commit and push the documentation separately.

**For now, complete Steps 1-3 and tell me what output you get.**

*10/9/2026, 10:14:16 AM · gpt-6*

```
PS C:\Users\POOJITHA
LAKSHMI\OneDrive\Desktop\23_dots_and_boxes> python -m
unittest discover -s tests -v
test_box_completion
(test_game.TestDotsAndBoxes.test_box_completion) ... ok
test_end_of_game_condition
(test_game.TestDotsAndBoxes.test_end_of_game_condition)
... ok
test_invalid_and_repeated_moves
(test_game.TestDotsAndBoxes.test_invalid_and_repeated_move
... ok
test_valid_horizontal_move
(test_game.TestDotsAndBoxes.test_valid_horizontal_move)
... ok
test_valid_vertical_move
(test_game.TestDotsAndBoxes.test_valid_vertical_move) ...
ok

----------------------------------------------------------------------
Ran 5 tests in 0.001s

OK
PS C:\Users\POOJITHA
LAKSHMI\OneDrive\Desktop\23_dots_and_boxes> git add
tests/test_game.py
>> git commit -m "Add automated game rule tests"
>> git push origin main
[main 28253df] Add automated game rule tests
 1 file changed, 62 insertions(+)
 create mode 100644 tests/test_game.py
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 814 bytes | 814.00 KiB/s, done.
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1
local object.
To https://github.com/PoojithaBrpl/23_dots_and_boxes.git
   69919c1..28253df  main -> main
PS C:\Users\POOJITHA
LAKSHMI\OneDrive\Desktop\23_dots_and_boxes>
```

All five automated tests passed, and your Task 4 commit has been pushed successfully to GitHub. ✅

Your terminal confirms:

- `Ran 5 tests`
- `OK`
- Commit: `28253df` - `Add automated game rule tests`
- Push to `main` succeeded.

## Next - Document your changes

The README requires you to document the changes and important design decisions. We'll update `README.md`, then commit and push that documentation separately.

## Step 1 - Update `README.md`

Open `README.md` in VS Code. **Keep the existing content** and add the following section at the bottom of the file.

## Implementation Notes

### 1. Invalid Coordinate Handling

Updated `board.py` to validate line orientation and coordinates before modifying the board. Invalid orientations and out-of-range coordinates now raise a `ValueError`, preventing accidental changes to the wrong board position.

### 2. Play-Again Feature

Updated `game.py` to allow players to start a new match after the current game ends. Starting a new match resets the board, scores, and current player.

### 3. Input Validation

Strengthened input handling in `game.py` to reject malformed commands, unsupported orientations, non-integer coordinates, negative coordinates, out-of-range moves, and repeated lines. Invalid moves do not change the board or scores.

### 4. Automated Testing

Added `tests/test_game.py` using Python's built-in `unittest` framework. The tests cover:

- Valid horizontal moves
- Valid vertical moves
- Invalid and repeated moves
- Box completion
- End-of-game detection

Run the tests from the project root using:

```
python -m unittest discover -s tests -v
```

All five automated tests passed during validation.

### Design Decisions

The existing modular structure was preserved. Board state and display remain in `board.py`, game flow and scoring remain in `game.py`, and move validation and completed-box counting remain in `rules.py`. Automated tests use the standard library, so no third-party dependencies are required.

## Step 2 - Save and commit the documentation

Save `README.md` with **Ctrl + S**, then run:

This will be your separate documentation commit.

## Step 3 - Check the commit history

Run:

You should see your recent commits, including:

- `Fix invalid board coordinates`
- `Add play again feature`
- `Strengthen game input validation`
- `Add automated game rule tests`
- `Document game improvements and tests`

The order shown by Git will be newest first.

**After pushing the README, tell me "done".** Then we'll finish the final video and submission checklist, including verifying your fork and preparing the chat-history export.

*10/9/2026, 10:16:07 AM · gpt-6*

done

*10/9/2026, 10:17:10 AM*